SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Preksha Prakash	**Reg. No.:**	1922260048
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>.*
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

1. Project Vision and Stakeholder Identification

Project Vision

To design and build an enterprise-grade **Zero-Downtime Database Migration and Schema Evolution Platform** that seamlessly executes database schema changes while maintaining 100% application availability. The platform automates pre-deployment schema validation, enforces backward-compatible migration patterns (e.g., expand-contract strategy), coordinates multi-version application compatibility, integrates directly with CI/CD pipelines, provides automated rollback mechanisms, and monitors real-time database performance to eliminate downtime and reduce deployment risks.

Project Objectives

- Validate database schema changes and detect breaking compatibility issues before execution.
- Support backward-compatible migration strategies across multiple application versions.
- Provide automated, single-click, or trigger-based rollback and recovery mechanisms.
- Seamlessly integrate migration workflows into existing CI/CD pipelines.
- Real-time monitoring and alerting of database health during active migrations.

Key Stakeholders & Roles

Stakeholder Role	Key Responsibilities & Interests
DevOps / SRE Engineer	Primary operational user; responsible for pipeline integration, automated rollbacks, deployment monitoring, and ensuring zero downtime.
Database Administrator (DBA)	Governs schema safety, reviews validation checks, sets data evolution policies, and audits database health.
Backend Software Engineer	Authors migration scripts, ensures code matches backward-compatible schema versions, and tests service compatibility.
Product Owner (PO)	Prioritizes backlog features, defines acceptance criteria, and ensures business value alignment.
Agile Lead / Scrum Master	Facilitates sprint events, removes development blockers, and enforces Agile practices.

2. Epics and User Stories

Epic 1: Schema Validation & Compatibility Analysis

Focuses on pre-execution checks, linting SQL/DDL scripts, and detecting destructive or non-backward-compatible schema changes.

- **US-1.1 (Validation Engine):**

As a Backend Engineer, *I want* to run automated schema compatibility checks on my pull requests, *so that* I can detect breaking changes before deploying to production.

- **US-1.2 (Backward Compatibility Rules):**

As a Database Administrator, *I want* the system to flag unsafe operations (e.g., immediate column drops, non-nullable additions without defaults), *so that* zero-downtime execution is guaranteed.

Epic 2: Zero-Downtime Execution & Strategy Engine

Enables progressive multi-phase migrations (e.g., Expand/Contract) while supporting dual application version reads/writes.

- **US-2.1 (Expand-Contract Migration Execution):**

As a DevOps Engineer, *I want* the platform to execute multi-step migrations without locking tables, *so that* active application instances continue functioning uninterrupted.

- **US-2.2 (Multi-Version Application Coordination):**

As a Backend Engineer, *I want* the migration engine to notify application instances of current schema versions, *so that* legacy and new app versions run concurrently during updates.

Epic 3: Rollback & Recovery Mechanisms

Provides automated safety nets and immediate failure recovery strategies during deployment failures.

- **US-3.1 (Automated Migration Rollback):**

As an SRE Engineer, *I want* the platform to automatically trigger a schema rollback upon detecting migration errors, *so that* system availability is restored instantly.

Epic 4: CI/CD Integration & Pipeline Automation

Provides CLI/API tools to seamlessly embed database schema evolution into modern deployment workflows.

- **US-4.1 (CI/CD Pipeline Plugin):**

As a DevOps Engineer, *I want* to integrate the platform as a step in my GitHub Actions / GitLab CI pipeline, *so that* migrations execute automatically upon release deployment.

Epic 5: Health Monitoring & Execution Audit

Real-time operational observability, lock tracking, and migration audit logs.

- **US-5.1 (Real-Time Health Monitoring Dashboard):**

As a DBA, *I want* to monitor active migration metrics (query latency, table lock wait times, replica lag), *so that* I can ensure database health remains stable during execution.

3. Acceptance Criteria (Given-When-Then Format)

US-1.1: Automated Schema Compatibility Checks

- **Scenario 1: Successful validation of a backward-compatible schema**
    - **Given** a developer submits a DDL script that adds a new nullable column user_status,
    - **When** the schema validation pipeline executes,
    - **Then** the platform should mark the check as "Passed" and report zero breaking risks.
- **Scenario 2: Rejection of a destructive non-backward-compatible change**
    - **Given** a developer submits a DDL script that drops a column currently in active use,
    - **When** the schema validation pipeline executes,
    - **Then** the check should fail with a severity warning: *"Destructive action detected: Column drop violates backward compatibility."*

US-2.1: Expand-Contract Execution Strategy

- **Scenario 1: Non-blocking table alter during live traffic**
    - **Given** an active application generating continuous database traffic,
    - **When** a migration adding an index or new column is executed,
    - **Then** table lock duration must remain under 50ms and application HTTP error rate must stay at 0%.

US-3.1: Automated Failure Detection and Rollback

- **Scenario 1: Auto-rollback on migration timeout/error**
    - **Given** a schema migration encounters an error or timeout during execution,
    - **When** the migration failure threshold is breached,
    - **Then** the system automatically executes the inverse rollback script, logs the anomaly, and sends an alert to Slack/PagerDuty.

US-4.1: CI/CD Execution Step

- **Scenario 1: Pipeline block on validation failure**
    - **Given** a pull request contains invalid migration syntax,
    - **When** the CI workflow triggers,
    - **Then** the pipeline stage aborts, preventing deployment to staging or production.

US-5.1: Real-Time Migration Health Tracking

- **Scenario 1: Threshold alert trigger**
    - **Given** a migration running on a production node,
    - **When** replica lag exceeds 5 seconds during data copying,
    - **Then** the monitoring dashboard highlights the node in Red and pauses step progression.

4. Product Backlog Prioritization & Sprint Planning

Prioritized Product Backlog

Backlog ID	Story Title	Business Value	Risk / Complexity	Priority	Target Sprint
US-1.1	Automated Schema Compatibility Checks	High	Low	**P1 (Highest)**	Sprint 1
US-1.2	Backward Compatibility Rule Engine	High	Medium	**P1**	Sprint 1
US-3.1	Automated Migration Rollback Engine	High	High	**P1**	Sprint 1
US-2.1	Expand-Contract Migration Execution	High	High	**P2**	Sprint 1
US-4.1	CI/CD Pipeline Plugin Integration	Medium	Medium	**P2**	Sprint 2
US-2.2	Multi-Version App Coordination	Medium	High	**P3**	Sprint 2
US-5.1	Real-time Health Monitoring Dashboard	Medium	Low	**P3**	Sprint 2

Sprint 1 Backlog & Actionable Task Breakdown

Selected User Stories for Sprint 1:

1. **US-1.1:** Automated Schema Compatibility Checks

2. **US-1.2:** Backward Compatibility Rule Engine
3. **US-3.1:** Automated Migration Rollback Engine
4. **US-2.1:** Expand-Contract Migration Execution Core

Actionable Task Breakdown:

- **Task 1.1 (US-1.1):** Build AST parser for DDL scripts to extract schema modifications.
- **Task 1.2 (US-1.2):** Implement linting rule set (detect dropped tables, renamed columns, non-default non-null additions).
- **Task 2.1 (US-2.1):** Build execution wrapper supporting non-blocking DDL operations (e.g., Postgres CONCURRENTLY or gh-ost strategy).
- **Task 3.1 (US-3.1):** Write transactional rollback executor to apply down-migrations upon error detection.
- **Task 3.2 (US-3.1):** Develop integration test suite simulating migration crashes and auto-recovery.

5. Story Point Estimation, Sprint Goal & Expected Deliverables

Story Point Estimation (Fibonacci Scale) & Justification

Story ID	Story Title	Story Points	Estimation Justification
US-1.1	Automated Schema Compatibility Checks	**3**	Straightforward DDL parsing logic; low technical risk.
US-1.2	Backward Compatibility Rule Engine	**5**	Requires defining multiple static analysis rules and error classification logic.
US-2.1	Expand-Contract Migration Execution	**8**	High complexity; involves live table modification, locks management, and concurrency safety.
US-3.1	Automated Migration Rollback Engine	**5**	Moderate complexity; requires inverse DDL generation and state tracking.
Total	**Sprint 1 Commitment**	**21 Points**	Matches standard 4-developer velocity over a 2-week sprint cycle.

Expected Deliverables at End of Sprint

1. **Schema Linter Engine:** Executable CLI tool that parses DDL files and reports breaking compatibility warnings.

2. **Zero-Downtime Executor Core:** Migration module supporting shadow table creation and non-blocking index additions.
3. **Rollback Handler:** Automated fallback mechanism triggering safe rollback scripts upon execution failures.
4. **Automated Test Suite:** Integration test demonstrating zero lost queries during simulated live migrations.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.

Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	**Mark**
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25